Legacy Modernization Orchestrator V1 Architecture

1. Product Goal

Legacy Modernization Orchestrator V1 is a local-first AI-human orchestration tool that helps migrate old React projects step by step into a modern React + TypeScript codebase.

The goal of V1 is not to perform a full automatic rewrite. The goal is to safely reduce manual migration effort by:

- Scanning an existing React project
- Detecting migration risks
- Creating a step-by-step migration plan
- Creating an isolated migration workspace
- Executing one migration step at a time
- Showing code diffs before approval
- Running validation after each step
- Allowing rollback when something fails
- Keeping the original project untouched

The core principle of the product is:

> AI executes. Human approves. Git protects.

V1 must feel like a production-grade developer tool, not a demo. The app should be fast, safe, predictable, and transparent.

---

2. Supported Project Types

V1 Supported

V1 should support only old React frontend projects.

Supported project types:

- React 16 projects
- React 17 projects
- JavaScript-based React projects
- JSX-based React components
- CRA-based React projects
- Vite-based React projects
- Webpack-based React projects
- npm, yarn, or pnpm package managers
- Single frontend repository
- Basic React Router usage

- Class components and functional components
- PropTypes-based components
- JavaScript utility files
- Basic Redux or Context-based state management

V1 Target Migration Direction

V1 migration should focus on:

- JavaScript to TypeScript migration
- JSX to TSX migration
- Adding TypeScript configuration
- Gradual typing of utilities and components
- Modernizing selected React patterns
- Fixing deprecated lifecycle methods where possible
- Improving project structure only when required
- Running validation after each step

Not Supported in V1

V1 should not support:

- Angular migration
- AngularJS migration
- Vue migration
- jQuery migration
- Backend migration
- Database migration
- COBOL, Java Swing, or desktop legacy migration
- Full monorepo migration
- Microfrontend migration
- Complete UI redesign
- Fully automatic production deployment
- Guaranteed 100% behavioral parity
- Business logic rewriting without human review

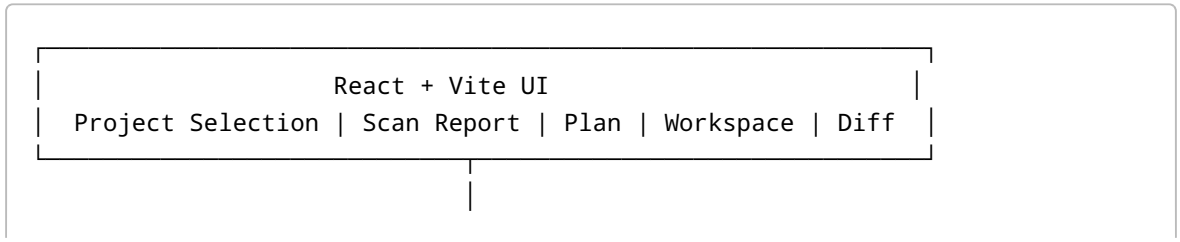
V1 should stay narrow and reliable.

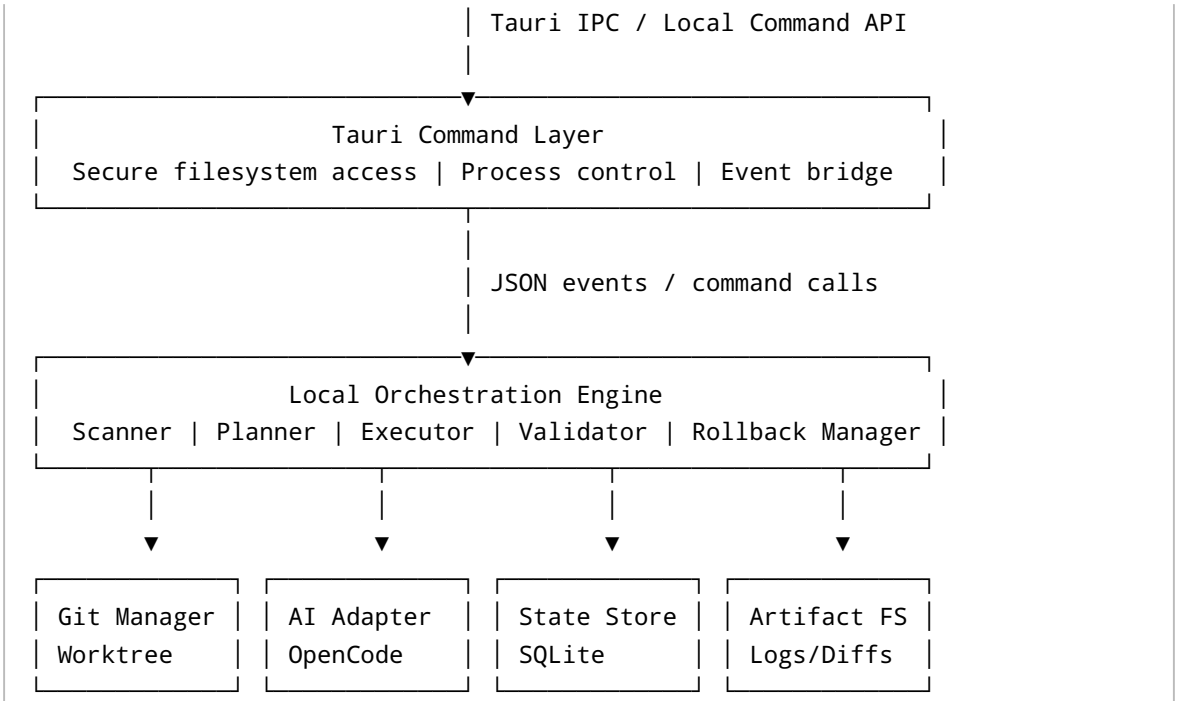
---

3. High-Level System Architecture

V1 should be a local-first desktop/web application.

Recommended architecture:





Recommended V1 Runtime Model

Use:

- React + Vite + TypeScript for the UI
- Tailwind CSS for styling
- Tauri for local desktop shell and secure OS integration
- Rust/Tauri command layer for controlled filesystem and process access
- Python sidecar or Node worker for orchestration logic
- SQLite for local session state
- File system artifact store for logs, prompts, diffs, plans, and reports
- Git worktree for isolated migration workspace
- OpenCode CLI or provider CLI as the first AI execution provider

Why This Architecture

This architecture gives:

- Fast local execution
- No dependency on cloud backend
- Strong project safety
- Better privacy for source code
- Clear separation between UI and execution
- Better control over AI-generated changes
- Simple packaging path for desktop usage
- Easier debugging through local logs and artifacts

---

4. Main Modules

4.1 UI Module

The UI should be focused, fast, and workflow-driven.

Main screens:

1. Project Selection
2. Scan Progress
3. Scan Report
4. Migration Plan
5. Create Workspace Confirmation
6. Execution Dashboard
7. Diff Review
8. Step Validation Result
9. Migration Summary

UI responsibilities:

- Show workflow progress
- Show scan results
- Show migration plan
- Ask for human approvals
- Stream execution logs
- Show file diffs
- Show validation results
- Show error recovery options
- Show token/cost usage
- Show current session state

Recommended UI libraries:

- React
- TypeScript
- Vite
- Tailwind CSS
- Zustand for local UI state
- TanStack Query for async command/query state
- React Virtual or TanStack Virtual for large logs and file lists
- Monaco Editor or CodeMirror for diff/code viewing
- React Hook Form for forms
- Zod for schema validation

Performance rules for UI:

- Do not load entire repository data into React state
- Virtualize long logs
- Virtualize large file lists
- Stream execution events incrementally
- Keep large diff content lazy-loaded

- Use memoization only for expensive calculations
- Avoid unnecessary global state
- Keep session data normalized
- Avoid re-rendering the full page on log updates
- Use route-level code splitting if screens grow large

4.2 Tauri Command Layer

The Tauri command layer should act as a secure bridge between the frontend and local machine.

Responsibilities:

- Select project folder
- Validate path access
- Start scan
- Start migration session
- Create Git worktree
- Start execution step
- Cancel execution step
- Read logs
- Read artifacts
- Read diffs
- Run validation commands
- Open workspace in editor
- Stream backend events to UI

Security rules:

- Do not expose arbitrary shell execution to the frontend
- Commands must be allowlisted
- File access must be scoped to selected project and generated workspace
- Do not allow the UI to pass raw shell commands
- Validate all paths before use
- Block path traversal
- Never send `.env` files or secrets to AI providers
- Redact sensitive values from logs and prompts

4.3 Orchestration Engine

The orchestration engine is the brain of the product.

Responsibilities:

- Maintain session lifecycle
- Control execution workflow
- Call scanner
- Generate migration plan
- Call AI provider

- Apply step execution
- Capture diffs
- Run validations
- Manage approvals
- Manage rollback
- Persist events
- Store artifacts

The orchestration engine should use a state machine, not random boolean flags.

Recommended session states:

```
DRAFT
PROJECT_SELECTED
SCANNING
SCAN_COMPLETED
PLAN_GENERATED
PLAN_APPROVED
WORKSPACE_CREATING
WORKSPACE_READY
EXECUTING_STEP
STEP_EXECUTED
AWAITING_DIFF_REVIEW
VALIDATING
STEP_APPROVED
STEP_COMMITTED
STEP_FAILED
ROLLING_BACK
ROLLED_BACK
COMPLETED
CANCELLED
```

Recommended step states:

```
PENDING
READY
RUNNING
AI_COMPLETED
DIFF_READY
VALIDATION_RUNNING
VALIDATION_PASSED
VALIDATION_FAILED
WAITING_FOR_APPROVAL
APPROVED
REJECTED
FIX_REQUESTED
COMMITTED
ROLLED_BACK
```

```
FAILED
SKIPPED
```

4.4 Scanner Module

The scanner module analyzes the selected React project before migration starts.

Scanner responsibilities:

- Detect package manager
- Detect React version
- Detect build tool
- Detect TypeScript presence
- Detect routing library
- Detect state management libraries
- Detect test setup
- Detect lint setup
- Detect build scripts
- Detect source folder
- Detect component files
- Detect utility files
- Detect class components
- Detect lifecycle methods
- Detect PropTypes
- Detect JavaScript/JSX file count
- Detect TypeScript/TSX file count
- Detect risky dependencies
- Detect migration complexity
- Generate risk score
- Generate migration recommendations

Scanner performance strategy:

- Ignore `node_modules`, `dist`, `build`, `.git`, `coverage`, and generated folders
- Use fast file globbing
- Use file hashing for cache
- Use AST parsing only where needed
- Do cheap metadata scan first
- Do deeper AST scan only after project is accepted
- Cache scan results per project path and Git commit SHA
- Avoid reading large binary files
- Avoid sending full source code to AI during scan

Scanner output:

```
scan-report.json
dependency-summary.json
file-inventory.json
```

```
risk-summary.json
migration-readiness.json
```

4.5 Migration Planner Module

The planner converts scan results into a safe migration plan.

Planner responsibilities:

- Generate step-by-step migration plan
- Prioritize low-risk foundation work first
- Separate migration into small reviewable steps
- Assign risk level to every step
- Identify files likely to change
- Define validation command per step
- Define rollback boundary per step
- Estimate effort and complexity
- Allow human edit before approval

Recommended V1 migration strategy:

```
1. Add TypeScript configuration
2. Add TypeScript dependencies
3. Add base type declarations
4. Convert simple utility files
5. Convert shared constants/config files
6. Convert simple stateless UI components
7. Convert reusable components
8. Convert form components
9. Convert route-level pages
10. Convert class components
11. Fix deprecated lifecycle methods
12. Update React Router usage if needed
13. Run final validation
14. Generate migration summary
```

Important rule:

> Planner should not create large steps. Every step should be small enough for a human to review comfortably.

4.6 Git Workspace Manager

The Git Workspace Manager protects the original project.

Responsibilities:

- Validate Git repository
- Check base branch
- Check clean working tree
- Create migration branch
- Create Git worktree
- Store workspace path
- Detect branch conflicts
- Detect existing workspace path
- Commit approved steps
- Roll back failed steps
- Remove workspace if user cancels

Recommended branch naming:

```
migration/react-ts/session-001
migration/react-ts/session-002
```

Recommended workspace path:

```
~/LegacyModernizer/workspaces/{project-name}/{session-id}
```

Safety rules:

- Original project must never be modified by AI execution
- All changes must happen inside the worktree
- Create workspace only after explicit human approval
- Stop if original repo has uncommitted changes
- Stop if base branch does not exist
- Stop if workspace path already exists
- Stop if branch already exists unless user explicitly chooses reuse
- Commit only after human approval
- Keep one commit per approved migration step

4.7 AI Provider Strategy

V1 should use a provider adapter pattern.

Recommended interface:

```ts
interface AIProvider {
  id: string;
  name: string;
  runTask(input: AITaskInput): Promise<AITaskResult>;
  streamTask(input: AITaskInput): AsyncIterable<AIEvent>;
```

```
  estimateCost?(input: AITaskInput): Promise<CostEstimate>;
}
```

V1 providers:

- OpenCode CLI as primary execution provider
- Manual prompt export as fallback
- Future support for Cursor CLI
- Future support for direct API providers

AI task types:

```
PROJECT_SUMMARY
SCAN_INTERPRETATION
PLAN_GENERATION
STEP_IMPLEMENTATION
AI_REVIEW
AUTO_FIX
VALIDATION_ANALYSIS
SUMMARY_GENERATION
```

Model routing strategy:

- Use stronger reasoning model for migration planning
- Use cost-effective model for simple code conversion
- Use stronger model for AI review
- Use smaller model for summarization and log analysis
- Avoid using expensive models for every step

Prompt strategy:

- Never send full repository blindly
- Send only relevant files
- Send scan summary
- Send current migration step
- Send exact constraints
- Send expected output format
- Send changed-file context only when required
- Keep prompts deterministic
- Store every prompt and response as artifacts

AI safety rules:

- AI must not modify original project path
- AI must work only inside migration workspace
- AI must not run destructive commands
- AI must not delete files unless the step explicitly allows it
- AI must not change unrelated files
- AI must explain changed files after execution
- AI output must be reviewed by diff before commit

4.8 Validation Module

The validation module determines whether a migration step is safe enough for review or commit.

Validation levels:

Level 1: Fast Validation

Runs after each step.

Examples:

```
npm run typecheck
npm run lint
npm test -- --watch=false
```

Level 2: Build Validation

Runs after important milestones.

Examples:

```
npm run build
```

Level 3: Final Validation

Runs before migration summary.

Examples:

```
npm run typecheck
npm run lint
npm test
npm run build
```

Validation detection:

- Detect available scripts from `package.json`
- Prefer existing project scripts
- Do not invent commands blindly
- If no validation script exists, show validation as unavailable
- Allow user to configure validation command manually

Validation output:

```
validation-result.json
validation-log.txt
error-summary.json
```

Validation status:

```
NOT_CONFIGURED
QUEUED
RUNNING
PASSED
FAILED
CANCELLED
```

4.9 Migration Session State

Session state should be persisted locally.

Recommended storage:

- SQLite for structured state
- File system for large artifacts

Recommended local data path:

```
~/.legacy-modernizer/
  app.db
  sessions/
    session-001/
      scan-report.json
      migration-plan.json
      execution-events.ndjson
      steps/
        step-001/
          prompt.md
          ai-response.md
          changed-files.json
          patch.diff
          validation-log.txt
          validation-result.json
```

Recommended SQLite tables:

```
projects
sessions
scan_reports
```

```
migration_plans
migration_steps
execution_events
artifacts
approvals
validation_runs
git_commits
provider_runs
settings
```

State persistence rules:

- Every important action should create an event
- Events should be append-only
- UI should recover from app restart
- Running session should resume as recoverable or failed
- Large logs should not be stored directly in React state
- Artifacts should be file-based and referenced from SQLite
- Step execution should be idempotent where possible

4.10 Error and Rollback Strategy

The product must treat failure as a normal workflow state.

Common error states:

```
Git worktree creation failed
Branch already exists
Workspace path already exists
Base branch not found
Original repo has uncommitted changes
Permission denied
AI provider failed
AI provider timed out
Validation failed
Build failed
Tests failed
Diff is too large
Unexpected files changed
Rollback failed
```

Error handling rules:

- Show clear error reason
- Show technical details separately
- Never hide command logs
- Suggest safe recovery actions
- Do not auto-commit failed changes

- Do not continue to next step after failure
- Allow retry when safe
- Allow rollback when changes exist
- Allow user to open workspace manually

Rollback strategy:

Before Commit

If step failed before approval:

```
git reset --hard
git clean -fd
```

Inside the migration workspace only.

After Commit

If an approved step needs rollback:

```
git revert <step-commit>
```

or

```
git reset --hard <previous-safe-commit>
```

depending on user choice.

Recommended V1 rollback behavior:

- Before commit: hard reset workspace to previous state
- After commit: create revert commit
- Never rollback original project
- Always show rollback summary

4.11 Execution Workflow

Recommended V1 workflow:

```
1. User selects project
2. System validates repository
3. System scans project
4. System shows scan report
5. System generates migration plan
6. User reviews and approves plan
```

```
7. System shows workspace confirmation
8. User approves workspace creation
9. System creates migration branch
10. System creates Git worktree
11. System starts first migration step
12. AI executes step inside workspace
13. System captures changed files
14. System generates diff
15. System runs validation
16. User reviews diff and validation result
17. User approves, rejects, fixes, or skips
18. System commits approved step
19. System moves to next step
20. Final validation runs
21. System generates migration summary
```

Human approval gates:

```
Approve migration plan
Approve workspace creation
Approve each step diff
Approve commit
Approve final summary
```

Optional human actions:

```
Edit plan
Skip step
Request AI fix
Reject step
Rollback step
Open workspace in editor
Run validation again
Cancel session
```

4.12 Performance Strategy

The app should be designed as a production-grade local tool.

App Startup

- Keep startup lightweight
- Do not scan projects on app launch
- Lazy-load heavy screens
- Load recent sessions only as metadata
- Do not load old logs until opened

Scanning Performance

- Use ignore rules aggressively
- Avoid scanning `node_modules`
- Cache by file hash and Git commit SHA
- Split scan into quick scan and deep scan
- Show scan progress incrementally
- Allow cancellation

Execution Performance

- Stream AI/provider output live
- Stream validation logs live
- Do not block UI during execution
- Run one migration step at a time in V1
- Avoid parallel AI writes to the same workspace
- Use bounded concurrency for read-only scanning
- Keep process output in append-only log files

Diff Performance

- Load diff file list first
- Load individual file diff on demand
- Virtualize large diffs
- Collapse unchanged sections
- Warn if diff is too large
- Allow file-level review

State Performance

- Use SQLite for structured state
- Use WAL mode for better local read/write behavior
- Store large logs as files
- Use JSONL/NDJSON for streaming event logs
- Debounce frequent UI updates
- Batch event writes where safe

UI Performance

- Keep global state small
- Use screen-level state where possible
- Use TanStack Query caching for backend calls
- Use Zustand for lightweight UI state
- Use virtualization for long lists
- Avoid unnecessary re-renders from log streaming
- Memoize expensive derived values only when measured

4.13 Security and Privacy Strategy

V1 should be safe by default.

Security principles:

- Local-first execution
- No cloud backend required
- No login required for V1
- Original project remains untouched
- AI execution happens only inside workspace
- User approves before mutation
- User approves before commit
- Commands are allowlisted
- File access is scoped
- Secrets are redacted

Sensitive files that should not be sent to AI:

```
.env
.env.local
.env.production
.env.development
*.pem
*.key
*.p12
*.crt
id_rsa
id_ed25519
secrets.*
credentials.*
```

The app should detect these files and exclude them from AI context by default.

4.14 Recommended Folder Structure

Recommended app structure:

```
legacy-modernization-orchestrator/
  apps/
    desktop/
      src/
        app/
        routes/
        components/
        features/
          project-selection/
          scanner/
          report/
          migration-plan/
          workspace/
          execution/
```

```
        diff-review/
        summary/
      shared/
        ui/
        hooks/
        utils/
        types/
    src-tauri/
      src/
        commands/
        security/
        events/
        process/
        filesystem/
        main.rs

orchestrator/
  core/
    session_manager.py
    workflow_engine.py
    event_bus.py

  scanner/
    project_scanner.py
    dependency_scanner.py
    ast_scanner.py
    risk_analyzer.py

  git/
    git_manager.py
    worktree_manager.py
    branch_manager.py
    rollback_manager.py

  ai/
    provider_base.py
    opencode_provider.py
    prompt_builder.py
    response_parser.py
    cost_tracker.py

  planner/
    migration_plan_generator.py
    step_builder.py
    risk_classifier.py

  validation/
    command_detector.py
    validation_runner.py
    validation_parser.py
```

```
    state/
      db.py
      repositories/
      migrations/

    artifacts/
      artifact_writer.py
      log_writer.py
      diff_writer.py

  docs/
    architecture/
    product/
    prompts/

  tests/
    scanner/
    git/
    workflow/
    validation/
```

4.15 V1 Limitations

V1 should clearly communicate its limitations.

Limitations:

- Supports only old React frontend projects
- Does not guarantee fully automatic migration
- Does not guarantee business logic parity
- Does not replace human code review
- Does not support Angular, Vue, jQuery, Java, or COBOL in V1
- Does not support complex monorepo migration in V1
- Does not automatically deploy migrated code
- Does not automatically merge PRs
- Does not solve missing test coverage
- Does not understand every custom architecture perfectly
- Does not guarantee zero AI mistakes

The product should be honest:

> This tool reduces migration effort. It does not remove engineering judgment.

4.16 V1 Success Criteria

V1 is successful if it can:

- Scan an old React project safely
- Generate a useful migration report
- Generate a practical step-by-step migration plan
- Create a safe Git worktree workspace
- Execute small migration steps
- Show clear diffs
- Run validation commands
- Allow approval before commit
- Roll back failed steps
- Keep original project untouched
- Persist full session history
- Provide a final migration summary

Recommended V1 benchmark:

```
Input: React 16/17 JavaScript project
Output: Partially or fully migrated React + TypeScript project
Execution style: Step-by-step
Review style: Human approval after every meaningful change
Safety model: Git worktree + validation + rollback
```

Final V1 Architecture Decision

For V1, use this architecture:

```
React + Vite + TypeScript UI
        ↓
Tauri secure command layer
        ↓
Local orchestration engine
        ↓
Git worktree workspace
        ↓
AI provider adapter
        ↓
Validation runner
        ↓
SQLite state + file artifacts
```

This is the right V1 architecture because it is:

- Local-first

- Fast
- Safe
- Privacy-friendly
- Production-grade
- Easy to debug
- Easy to extend
- Narrow enough to build
- Strong enough to become a serious developer tool